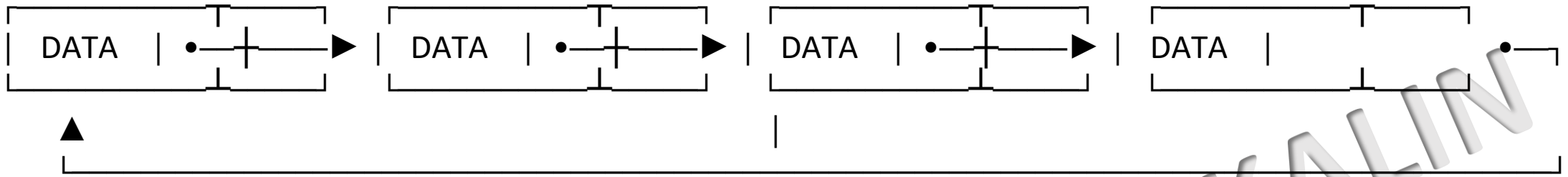


VERİ YAPILARI

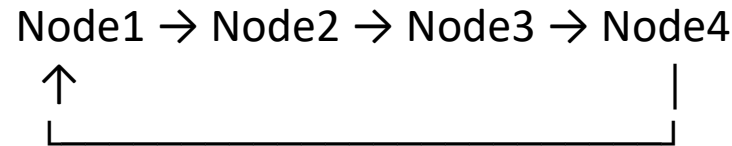
Dr. Öğr. Üyesi Fatma AKALIN

DAİRESEL TEK YÖNLÜ BAĞLI LİSTE

Verilere sıralı olarak erişebilmek için tasarlanan **bağlı listeler**, elemanlarının **fiziksel sıralı olması yerine mantıksal sıralı** oluşturulmaktadır. Tek yönlü sıralı doğrusal bir bağlı liste yapısında **aradaki** bir eleman üzerinde çalışırken **önceki** elemanlara **ulaşmak** ihtiyacı olduğunda liste başı işaretçisinden başlamak zorunludur. Çünkü listenin herhangi bir elemanından sadece kendisinden bir sonraki elemana işaretçi bulunmaktadır. Bu durumda eğer listede **çok sayıda eleman** varsa **herhangi bir elemana ulaşma maliyeti** böyle bir **bağlı liste için yüksek** olacaktır. Bağlı liste yapısına ilave bir özellik katılarak bir bağlı listede en son elemanın sonraki kısmının **NULL değeri göstermesi yerine listenin ilk elemanını göstermesi** sağlanırsa bu probleme çözüm üretilebilir. Listenin son elemanının liste başındaki elemana işaretçi bulundurduğu listelere ***dairesel tek yönlü bağlı liste*** denir.



Dairesel bağlı bir listede doğrusal listede olduğu gibi **herhangi bir yere eleman ekleme** ve **listenin herhangi bir yerinden eleman çıkarma** olarak işlevselliklerini gerçekleştirilebilir.



```

1 #include <iostream>
2 using namespace std;
3 struct node{
4     string kullanıcıadi;
5     int parola;
6     node * next;
7 };
8 int main() {
9     struct node * uye1=new node(); //5 tane düğüm oluşturma siteye giriş yapacak
10    struct node * uye2=new node();
11    struct node * uye3=new node();
12    struct node * uye4=new node();
13    struct node * uye5=new node();
14
15    uye1->kullanıcıadi="a1";
16    uye2->kullanıcıadi="a2";
17    uye3->kullanıcıadi="a3";
18    uye4->kullanıcıadi="a4";
19
20    uye1->parola=111;
21    uye2->parola=222;
22    uye3->parola=333;
23    uye4->parola=444;
24
25    uye1->next=uye2;
26    uye2->next=uye3;
27    uye3->next=uye4;
28    uye4->next=uye1;
29
30    cout<<"1. dugumun kullanıcı adı ve şifresi : "<<uye1->kullanıcıadi<<" & "<<uye1->parola<<" "<<endl;
31    cout<<"2. dugumun kullanıcı adı ve şifresi : "<<uye2->kullanıcıadi<<" & "<<uye2->parola<<endl;
32    cout<<"3. dugumun kullanıcı adı ve şifresi : "<<uye3->kullanıcıadi<<" & "<<uye3->parola<<endl;
33    cout<<"4. dugumun kullanıcı adı ve şifresi : "<<uye4->kullanıcıadi<<" & "<<uye4->parola<<endl;
34    cout<<uye4->next<<endl;
35    cout<<uye1<<endl;
36    return 0;
37 }

```

Tek tek düğüm oluşturup
tek yönlü dairesel bağlı bir
listede gösterme

```
C:\Users\Fatma\Desktop\VERİYAPILARI\kitap\daireselbagliliste>g++ -o ornek1 ornek1.cpp
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\kitap\daireselbagliliste>ornek1
```

```

1. dugumun kullanıcı adı ve şifresi : a1 & 111
2. dugumun kullanıcı adı ve şifresi : a2 & 222
3. dugumun kullanıcı adı ve şifresi : a3 & 333
4. dugumun kullanıcı adı ve şifresi : a4 & 444
0xef1da0
0xef1da0

```

```

1.cpp x ornek2.cpp x
#include <iostream>
using namespace std;

struct node{
    string kullanıcıadi;
    int parola;
    node * next;
};

int main(){
    struct node * uye1=new node(); //5 tane düğüm oluşturma siteye giris yapacak
    struct node * uye2=new node();
    struct node * uye3=new node();
    struct node * uye4=new node();
    struct node * uye5=new node();

    uye1->kullanıcıadi="a1";
    uye2->kullanıcıadi="a2";
    uye3->kullanıcıadi="a3";
    uye4->kullanıcıadi="a4";

    uye1->parola=111;
    uye2->parola=222;
    uye3->parola=333;
    uye4->parola=444;

    uye1->next=uye2;
    uye2->next=uye3;
    uye3->next=uye4;
    uye4->next=uye1;

    struct node *temp=uye1;

    cout<<"1. dugumun kullanıcı adi ve şifresi : "<<temp->kullanıcıadi<<" & "<<temp->parola<<endl;
    temp=temp->next;
    int i=2;
    while (temp!=uye1){
        cout<<i<<" . dugumun kullanıcı adi ve şifresi : "<<temp->kullanıcıadi<<" & "<<temp->parola<<endl;
        temp=temp->next;
        i++;
    }
    return 0;
}

```

Tek tek düğüm oluşturup
döngü vasıtasıyla tek
yönlü dairesel bağlı bir
listede gösterme

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\daireselebagliliste>g++ -o ornek2 ornek2.cpp
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\daireselebagliliste>ornek2
```

```

1. dugumun kullanıcı adi ve şifresi :a1 & 111
2. dugumun kullanıcı adi ve şifresi :a2 & 222
3. dugumun kullanıcı adi ve şifresi :a3 & 333
4. dugumun kullanıcı adi ve şifresi :a4 & 444

```

```

.cpp x ornek2.cpp x ornek3.cpp x C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\daireselbagliliste>g++ -o ornek3 ornek3.cpp

#include <iostream>
using namespace std;

struct node{
    int kullaniciadi;
    int veri;
    node * next;
};

struct node * basdugum=NULL;

struct node * basaekle(node * basdugum, int kullaniciadi, int veri){
    struct node * nnode=new node();
    nnode->kullaniciadi=kullaniciadi;
    nnode->veri=veri;
    if(basdugum==NULL){
        nnode->next=nnode;
        basdugum=nnode;
    }
    else if(basdugum->next==basdugum){
        nnode->next=basdugum;
        basdugum->next=nnode;
        basdugum=nnode;
    }
    else{
        node * iter=basdugum;
        while(iter->next!=basdugum){
            iter=iter->next;
        }
        nnode->next=basdugum;
        basdugum->next=nnode;
        iter->next=basdugum;
    }
    return basdugum;
}

void yazdir(node * basdugum){
    if(basdugum==NULL){
        cout<<"Yazdirilacak eleman yok";
    }
    node * iter=basdugum;
    cout<<"kullanici adi "<<iter->kullaniciadi<<" Veri "<<iter->veri<<endl;
    iter=iter->next;
    while(iter!=basdugum){
        cout<<"kullanici adi "<<iter->kullaniciadi<<" Veri "<<iter->veri<<endl;
        iter=iter->next;
    }
}

int main(){
    int kullaniciadi;
    int veri;
    node * root=NULL;
    root=basaekle(root,1,1);
    root=basaekle(root,2,2);
    root=basaekle(root,3,3);
    root=basaekle(root,4,4);
    yazdir(root);
}

```

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\kitap\daireselbagliliste>ornek3
kullanici adi 4 Veri 4
kullanici adi 3 Veri 3
kullanici adi 2 Veri 2
kullanici adi 1 Veri 1

```

Dairesel bağlı bir listede başa eleman ekleyen fonksiyonu gösterme

```

struct node * sonaekle(struct node * basdugum, int kullanicadi, int
veri){
    struct node * nnode=new node();
    nnode->kullanicadi=kullanicadi;
    nnode->veri=veri;
    if(basdugum==NULL){
        nnode->next=nnode;
        basdugum=nnode;
    }
    else if(basdugum->next==basdugum){
        basdugum->next=nnode;
        nnode->next=basdugum;
    }
    else{
        node * iter=basdugum;
        while(iter->next!=basdugum){
            iter=iter->next;
        }
        iter->next=nnode;
        nnode->next=basdugum;
    }
    return basdugum;
}

```

Dairesel Bağlı Listede Sona Düğüm Ekleme Fonksiyonu

```
struct node * bastansil(node * basdugum){
    if(basdugum==NULL){
        cout<<"Liste bos";
    }
    else if(basdugum->next==basdugum){
        delete basdugum;
        basdugum=NULL;
    }
    else{
        node * iter=basdugum;

        while(iter->next!=basdugum){
            iter=iter->next;
        }
        node * stemp=basdugum->next;
        delete basdugum;
        basdugum=stemp;
        iter->next=basdugum;

    }
    return basdugum;
}
```

Dairesel Bağlı Listede Baştan Düğüm Silme Fonksiyonu

```
struct node * sondansil(node * basdugum){
    if(basdugum==NULL){
        cout<<"Liste bos";
    }
    else if(basdugum->next==basdugum){
        delete basdugum;
        basdugum=NULL;
    }
    else{
        node * iter=basdugum;
iter=iter->next;
        while(iter->next->next!=basdugum){
            iter=iter->next;
        }
        node * stemp=iter->next;
        delete stemp;
        iter->next=basdugum;
    }
    return basdugum;
}
```

Dairesel Bağlı Listede Sondan Düğüm Silme Fonksiyonu

C:\Users\Fatma\Desktop\VERİYAPILARI\kitap\daireselbagliliste>g++ -o ornek5 ornek5.cpp

C:\Users\Fatma\Desktop\VERİYAPILARI\kitap\daireselbagliliste>ornek5

kullanici adi 1 Veri 1

kullanici adi 2 Veri 2

kullanici adi 3 Veri 3

kullanici adi 4 Veri 4

Listedeki hangi kullaniciadina sahip elemandan once araya eleman eklemek istersiniz?4

kullanici adi 1 Veri 1

kullanici adi 2 Veri 2

kullanici adi 3 Veri 3

kullanici adi 5 Veri 5

kullanici adi 4 Veri 4

Listedeki hangi kullaniciadina sahip elemandan once araya eleman eklemek istersiniz?1

kullanici adi 1 Veri 1

kullanici adi 2 Veri 2

kullanici adi 3 Veri 3

kullanici adi 5 Veri 5

kullanici adi 4 Veri 4

kullanici adi 6 Veri 6

Listedeki hangi kullaniciadina sahip elemandan once araya eleman eklemek istersiniz?3

kullanici adi 1 Veri 1

kullanici adi 2 Veri 2

kullanici adi 7 Veri 7

kullanici adi 3 Veri 3

kullanici adi 5 Veri 5

kullanici adi 4 Veri 4

kullanici adi 6 Veri 6

1 Filtreleme İşlevleri

Bu örnekte:

Parola aralığı filtreleme

İsim başlangıcına göre filtreleme

Belirli uzunlukta parolaları yazdırma

```

#include <iostream>
using namespace std;

struct node {
    string isim;
    int parola;
    node* next;
};

// Düğüm ekleme
node* ekle(node* başdugum, string isim, int parola) {
    node* yeniDugum = new node{isim, parola, nullptr};
    if (!başdugum) { yeniDugum->next = yeniDugum; return yeniDugum; }
    node* iter = başdugum;
    while (iter->next != başdugum) iter = iter->next;
    iter->next = yeniDugum;
    yeniDugum->next = başdugum;
    return başdugum;
}

// Yazdır
void yazdir(node* başdugum) {
    if (!başdugum) { cout << "Liste bos." << endl; return; }
    node* iter = başdugum;
    do {
        cout << iter->isim << " -> " << iter->parola << endl;
        iter = iter->next;
    } while (iter != başdugum);
}

```

Eğer `başdugum` **true** ise `!başdugum` **false** olur.

Eğer `başdugum` **false** ise `!başdugum` **true** olur.

`new node{isim, parola, nullptr};`

şununla aynı anlama gelir:

`node* yeniDugum = new node;`

`yeniDugum->isim = isim;`

`yeniDugum->parola = parola;`

`yeniDugum->next = nullptr;`

// 1. Parola aralığı filtreleme

```
void parolaAraligi(node* başdugum, int alt, int ust) {  
    if (!başdugum) return;  
    node* iter = başdugum;  
    do {  
        if (iter->parola >= alt && iter->parola <= ust)  
            cout << iter->isim << " -> " << iter->parola << endl;  
        iter = iter->next;  
    } while (iter != başdugum);  
}
```

// 2. İsim başlangıcına göre filtreleme

```
void isimBaslangici(node* başdugum, char basHarf) {  
    if (!başdugum) return;  
    node* iter = başdugum;  
    do {  
        if (iter->isim[0] == basHarf)  
            cout << iter->isim << " -> " << iter->parola << endl;  
        iter = iter->next;  
    } while (iter != başdugum);  
}
```

IN

```
// 3. Belirli uzunlukta parolalar
void parolaUzunlugu(node* başdugum, int uzunluk) {
    if (!başdugum) return;
    node* iter = başdugum;
    do {
        int len = to_string(iter->parola).length();
        if (len == uzunluk)
            cout << iter->isim << " -> " << iter->parola << endl;
        iter = iter->next;
    } while (iter != başdugum);
}
```

Dr.

to_string() fonksiyonunun görevi:
Sayısal bir değeri string'e (yazıya) çevirmek.

```
int main() {  
    node* başdugum = NULL;  
    başdugum = ekle(başdugum, "Ali", 111);  
    başdugum = ekle(başdugum, "Ayşe", 250);  
    başdugum = ekle(başdugum, "Mehmet", 333);  
    başdugum = ekle(başdugum, "Fatma", 1234);  
  
    cout << "Parola 200-400 arasi:" << endl;  
    parolaAraligi(başdugum, 200, 400);  
  
    cout << "\nIsim A ile baslayanlar:" << endl;  
    isimBaslangici(başdugum, 'A');  
  
    cout << "\n3 haneli parolalar:" << endl;  
    parolaUzunlugu(başdugum, 3);  
  
    return 0;  
}
```

2 Düğüm Güncelleme İşlevleri

Belirli isim veya parolaya göre **parola veya isim güncelleme**

Dr. Öğr. Üyesi Fatma AKALIN

```
#include <iostream>
using namespace std;

struct node {
    string isim;
    int parola;
    node* next;
};

node* ekle(node* başdugum, string isim, int parola) {
    node* yeniDugum = new node{isim, parola, nullptr};
    if (!başdugum) { yeniDugum->next = yeniDugum; return yeniDugum; }
    node* iter = başdugum;
    while (iter->next != başdugum) iter = iter->next;
    iter->next = yeniDugum;
    yeniDugum->next = başdugum;
    return başdugum;
}

void yazdir(node* başdugum) {
    if (!başdugum) { cout << "Liste bos." << endl; return; }
    node* iter = başdugum;
    do {
        cout << iter->isim << " -> " << iter->parola << endl;
        iter = iter->next;
    } while (iter != başdugum);
}
```

KALIN

```
// Parolayı güncelle
void parolaGuncelle(node* başdugum, string isim, int yeniParola) {
    if (!başdugum) return;
    node* iter = başdugum;
    do {
        if (iter->isim == isim) {
            iter->parola = yeniParola;
            return;
        }
        iter = iter->next;
    } while (iter != başdugum);
}
```

```
// İsmi güncelle
void isimGuncelle(node* başdugum, int parola, string yeniIsim) {
    if (!başdugum) return;
    node* iter = başdugum;
    do {
        if (iter->parola == parola) {
            iter->isim = yeniIsim;
            return;
        }
        iter = iter->next;
    } while (iter != başdugum);
}
```

```
int main() {
    node* başdugum = NULL;
    başdugum = ekle(başdugum, "Ali", 100);
    başdugum = ekle(başdugum, "Ayşe", 250);

    cout << "Orijinal Liste:" << endl;
    yazdir(başdugum);

    cout << "\nAli'nin parolasi guncelleniyor..." << endl;
    parolaGuncelle(başdugum, "Ali", 999);
    yazdir(başdugum);

    cout << "\n250 parolali isim guncelleniyor..." << endl;
    isimGuncelle(başdugum, 250, "Fatma");
    yazdir(başdugum);

    return 0;
}
```

3 Liste Analiz İşlevleri

Toplam parolayı bulma

En büyük / en küçük parolayı bulma

Parola eşik kontrolü (500'ü geçenleri yazdır)

Dr. Öğr. Üyesi Fatma AKALIN

```
#include <iostream>
using namespace std;
```

```
struct node {
    string isim;
    int parola;
    node* next;
};
```

```
node* ekle(node* başdugum, string isim, int parola) {
    node* yeniDugum = new node{isim, parola, nullptr};
    if (!başdugum) { yeniDugum->next = yeniDugum; return yeniDugum; }
    node* iter = başdugum;
    while (iter->next != başdugum) iter = iter->next;
    iter->next = yeniDugum;
    yeniDugum->next = başdugum;
    return başdugum;
}
```

```
void yazdir(node* başdugum) {
    if (!başdugum) { cout << "Liste bos." << endl; return; }
    node* iter = başdugum;
    do {
        cout << iter->isim << " -> " << iter->parola << endl;
        iter = iter->next;
    } while (iter != başdugum);
}
```

```
// Toplam parolayı bul
int toplamParola(node* başdugum) {
    if (!başdugum) return 0;
    int toplam = 0;
    node* iter = başdugum;
    do {
        toplam += iter->parola;
        iter = iter->next;
    } while (iter != başdugum);
    return toplam;
}

// En büyük parola
int enBuyukParola(node* başdugum) {
    if (!başdugum) return -1;
    int maxVal = başdugum->parola;
    node* iter = başdugum->next;
    while (iter != başdugum) {
        if (iter->parola > maxVal) maxVal = iter->parola;
        iter = iter->next;
    }
    return maxVal;
}
```

ALIN



```
// En küçük parola
int enKucukParola(node* başdugum) {
    if (!başdugum) return -1;
    int minVal = başdugum->parola;
    node* iter = başdugum->next;
    while (iter != başdugum) {
        if (iter->parola < minVal) minVal = iter->parola;
        iter = iter->next;
    }
    return minVal;
}

// Parolası 500'ü geçenleri yazdır
void parola500Ustu(node* başdugum) {
    if (!başdugum) return;
    node* iter = başdugum;
    do {
        if (iter->parola > 500)
            cout << iter->isim << " -> " << iter->parola << endl;
        iter = iter->next;
    } while (iter != başdugum);
}
```



```
int main() {  
    node* başdugum = NULL;  
    başdugum = ekle(başdugum, "Ali", 100);  
    başdugum = ekle(başdugum, "Ayşe", 600);  
    başdugum = ekle(başdugum, "Mehmet", 450);  
  
    cout << "Liste:" << endl;  
    yazdir(başdugum);  
  
    cout << "\nToplam parola: " << toplamParola(başdugum) << endl;  
    cout << "En büyük parola: " << enBuyukParola(başdugum) << endl;  
    cout << "En küçük parola: " << enKucukParola(başdugum) << endl;  
  
    cout << "\nParolasi 500'u geçenler:" << endl;  
    parola500Ustu(başdugum);  
  
    return 0;  
}
```

Kaynakça

Veri Yapıları ve Algoritmalar, 12. Baskı, Rifat Çölkesen

Nejat Yumuşak, fatih Adak Veri C++ ile veri yapıları, 3.baskı

C++ ile programlama, Dokuzuncu baskıdan çeviri, Palme Yayınları, Pau Deitel,Harvey Deitel, Çeviri Editörü Cemil Öz

Veri Yapıları, Hakan Kutucu

Algoritma ve Programlamaya giriş, Fahri Vatansever

Programlama ve Veri Yapılarına Giriş C,C++Cve Java ile, Sadi Evren Şeker